

STUDY GUIDE

Module 3

React Fundamentals

Table of Contents

(In Word: right-click below and choose "Update Field" to generate page numbers.)

Table of Contents	2
1. Introduction to React & The Virtual DOM	4
1.1 Paradigm Shifts in UI Development	4
1.1.1 Imperative Programming (Vanilla JS) vs. Declarative Architecture (React)	4
1.1.2 Component-Driven Development Philosophy	5
1.2 The Rendering Engine Under the Hood	5
1.2.1 The Browser Real DOM	5
1.2.2 The In-Memory Virtual DOM (VDOM) Representation	6
1.2.3 The React Fiber Reconciliation Engine & The Diffing Algorithm	7
2. JSX (JavaScript XML) Fundamentals	9
2.1 Compilation & Compilation Tools	9
2.1.1 The Role of Transpilers (Babel) in Converting JSX to React.createElement	9
2.1.2 The Modern JSX Runtime Transformation	9
2.2 Syntactic Strict Rules	10
2.2.1 The Single Root Element Constraint & React Fragments (<>...</>)	10
2.2.2 Mapping HTML Attributes to camelCase JavaScript Properties	10
2.2.3 Dynamic Evaluation: Evaluating JavaScript Expressions with Curly Braces {}	11
3. React Functional Components	12
3.1 Component Architecture	12
3.1.1 Defining Components as Functional Elements	12
3.1.2 Strict Purity Contracts: Idempotency & Avoiding Side Effects during Rendering	12
3.2 Modular Composition	13
3.2.1 Component Nesting, Isolation, and Reusability Models	13
4. Data Mechanics: Props & State	14
4.1 Props (Properties) Deep Dive	14
4.1.1 The Unidirectional Data Flow Model (Top-to-Bottom)	14
4.1.2 Read-Only Immutability Constraints of Props	14
4.1.3 Destructuring and Setting Default Props	15
4.2 State Management Foundations	15
4.2.1 Local Component Memory & The useState Hook Engine	15
4.2.2 Asynchronous State Updates & Batching Behaviors	16
4.2.3 Updating Objects and Arrays State Immutably	17

5. Interactive Event Handling & Conditional Layouts	18
5.1 Event System Architecture.....	18
5.1.1 Synthetic Events: Normalizing Browser Compatibility Cross-Platform.....	18
5.1.2 Passing Reference Handlers vs. Immediate Function Invocation Footguns	18
5.2 Dynamic Conditional Rendering Patterns.....	19
5.2.1 Inline Expressions: Ternary Operators (?:) for Binary Branches	19
5.2.2 Short-Circuit Operators (&&) for Short-Form Evaluation Guard Rails.....	19
6. Iterative Lists & The Key Constraint.....	20
6.1 Rendering Data Collections.....	20
6.1.1 Projecting Arrays to JSX Elements via .map().....	20
6.2 The Key Attribute Contract	20
6.2.1 Why Keys Matter: Optimizing the Virtual DOM Diffing Engine	20
6.2.2 The Array Index Anti-Pattern and Stable Identity Generation	21
7. Component Lifecycle Concepts & Coordination	22
7.1 The Core Lifecycle Phases	22
7.1.1 Mount (Initialization & DOM Insertion).....	22
7.1.2 Update (Prop/State Change Re-rendering).....	22
7.1.3 Unmount (DOM Clean-up & Destruction)	22
7.2 Lifecycle Synchronization.....	23
7.2.1 Managing Lifecycles using the useEffect Hook	23
7.2.2 The Dependency Array ([]): Control-Flow Triggers	23
7.2.3 Clean-up Callbacks: Preventing Memory Leaks and Subscriptions Drops	24

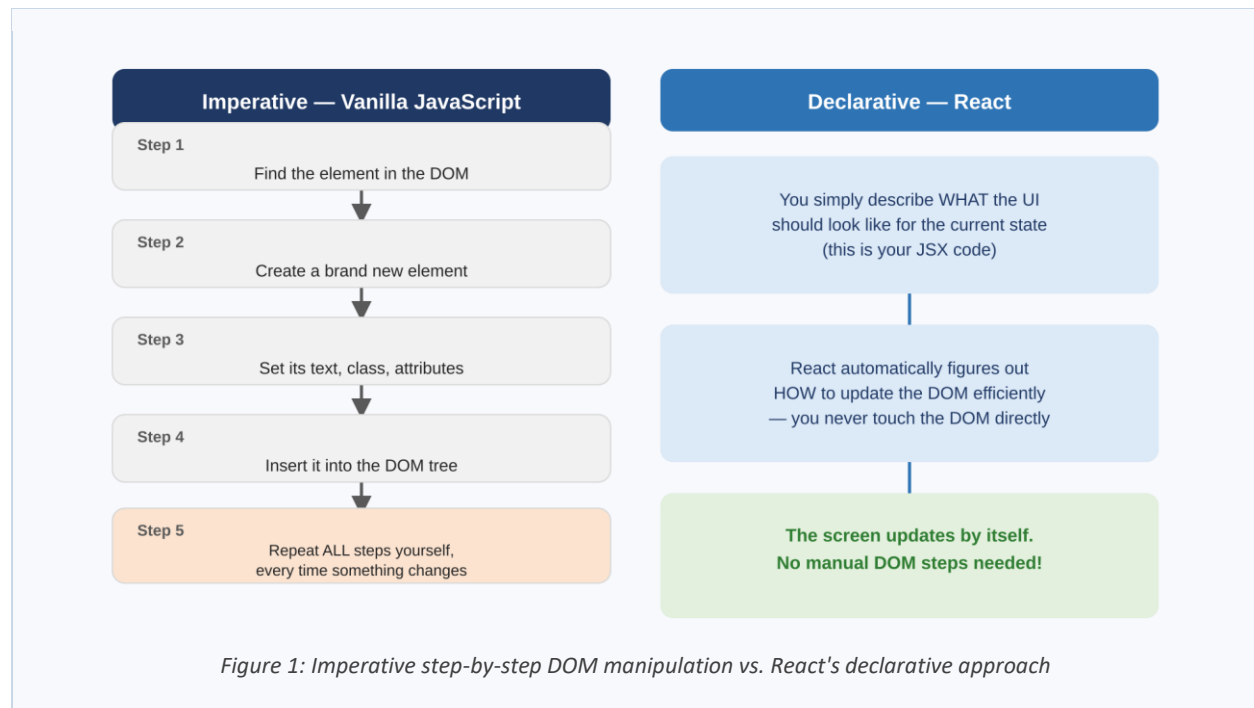
1. Introduction to React & The Virtual DOM

1.1 Paradigm Shifts in UI Development

1.1.1 Imperative Programming (Vanilla JS) vs. Declarative Architecture (React)

In imperative programming, you write step-by-step instructions telling the browser exactly HOW to change the screen: find this element, create that element, set its text, insert it here. You are responsible for every single step, every time something changes.

In declarative programming, which is how React works, you simply describe WHAT the UI should look like for a given piece of data. React itself figures out how to make the actual screen match your description, so you never touch the DOM by hand.



Example Syntax

```
// Imperative -> vanilla JavaScript
const heading = document.createElement('h1');
heading.textContent = 'Hello World';
heading.className = 'title';
document.body.appendChild(heading);
// If the text needs to change later, YOU must find it and update it again

// Declarative -> React
function Greeting() {
  return <h1 className='title'>Hello World</h1>;
}
```

```
// If the text needs to change, you just change the data.  
// React re-renders the description and updates the screen for you.
```

1.1.2 Component-Driven Development Philosophy

React encourages you to break a UI into small, independent, reusable pieces called components. Each component manages its own markup, logic, and (optionally) its own data, and can be combined with other components like building blocks to form a full page.

This makes large applications far easier to build, test, reuse, and maintain, because you only need to think about one small piece at a time.

Example Syntax

```
function Button({ label }) {  
  return <button>{label}</button>;  
}  
  
function Toolbar() {  
  return (  
    <div>  
      <Button label='Save' />  
      <Button label='Cancel' />  
    </div>  
  );  
}  
// Toolbar is built entirely out of smaller, reusable Button components
```

Beginner Tip: Think of components the way you think about LEGO bricks — small, self-contained pieces that snap together to build something bigger.

1.2 The Rendering Engine Under the Hood

1.2.1 The Browser Real DOM

The Real DOM (Document Object Model) is the browser's actual, live representation of your HTML page. Every tag becomes a 'node' in a tree structure, and the browser uses this tree to draw pixels on the screen.

The problem is that updating the Real DOM directly is relatively slow, especially if you do it often or change many elements at once, because the browser may need to recalculate layout and repaint the screen every time.

Example Syntax

```
// Directly touching the Real DOM, one change at a time  
document.getElementById('title').textContent = 'Updated!';
```

```
// Each direct change like this can force the browser to  
// recalculate layout and repaint part of the screen.
```

1.2.2 The In-Memory Virtual DOM (VDOM) Representation

The Virtual DOM is a lightweight copy of the Real DOM, but it exists only in memory as plain JavaScript objects, not as real screen elements. Because it is just JavaScript objects, creating and comparing versions of it is extremely fast.

Whenever your data (state) changes, React does not touch the Real DOM straight away. Instead, it first builds a brand new Virtual DOM tree describing what the UI should look like now.

Example Syntax

```
// A simplified idea of what a Virtual DOM 'element' looks like  
// (this is roughly what React.createElement produces)  
const virtualElement = {  
  type: 'h1',  
  props: { className: 'title', children: 'Hello World' },  
};  
// It is just a plain JavaScript object -> cheap to create and compare
```

ReactJS Virtual DOM is an in-memory representation of the actual DOM (Document Object Model). React uses this lightweight JavaScript object to track changes in the application state and efficiently update the actual DOM only where necessary.

- **Step 1 – Initial Rendering:** when the app starts, the entire UI is represented as a Virtual DOM. React elements are created and rendered into the virtual structure.
- **Step 2 – State and Props Changes:** as the states and props change in the app, React re-renders the affected components in the virtual DOM. These changes do not immediately impact the real DOM.
- **Step 3 – Comparison Using Diff Algorithm:** React then uses a diffing algorithm to compare the current version of the Virtual DOM with the previous version. This process identifies the differences (or "diffs") between the two versions.
- **Step 4 – Reconciliation Process:** based on the differences identified, React determines the most efficient way to update the real DOM. Only the parts of the real DOM that need to be updated are changed, rather than rerendering the entire UI. This selective updating is quick and performant.
- **Step 5 – Update to the Real DOM:** finally, React applies the necessary changes to the real DOM. This might involve adding, removing, or updating elements based on the differences detected in step 3.

How Virtual DOM works?

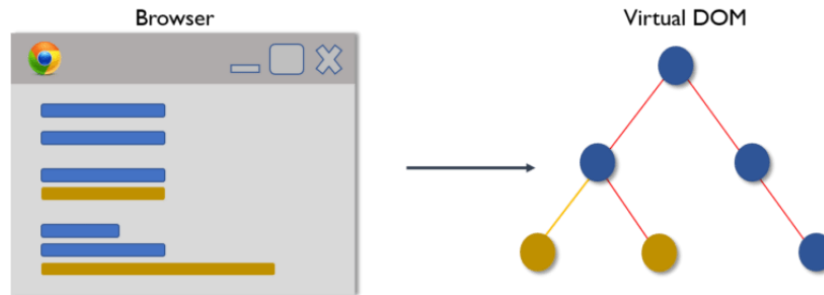
The Virtual DOM works in five simple steps.

1. Initial Render -

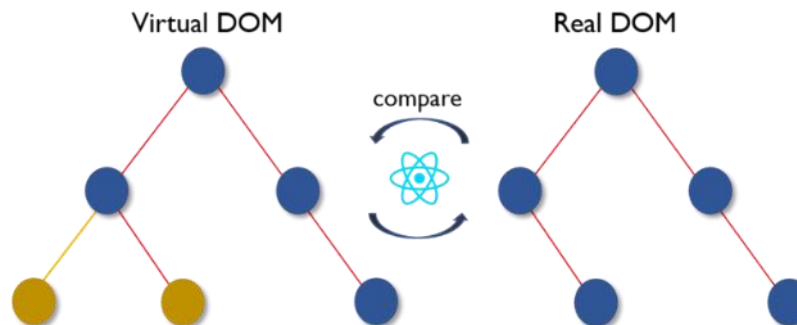
When a UI component renders for the first time, it returns JSX. React uses this structure to create a Virtual DOM tree, which is a lightweight copy of the actual DOM. This Virtual DOM is then used to build and render the Real DOM in the browser.

2. State or Props Change -

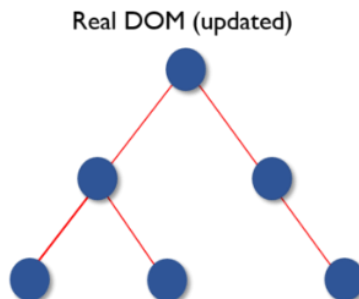
When the component's state or props change, React creates a new Virtual DOM reflecting the updated UI. However, it doesn't immediately update the Real DOM; instead, it works in memory to prepare for an efficient update.



3. Diffing Algorithm React then compares the new Virtual DOM with the previous one using a process called diffing. It determines what has changed between the two versions and identifies the minimal set of updates needed.



4. Reconciliation Based on the diffing results, React decides which parts of the Real DOM should be updated. It avoids re-rendering the entire DOM and instead updates only the elements that actually changed.

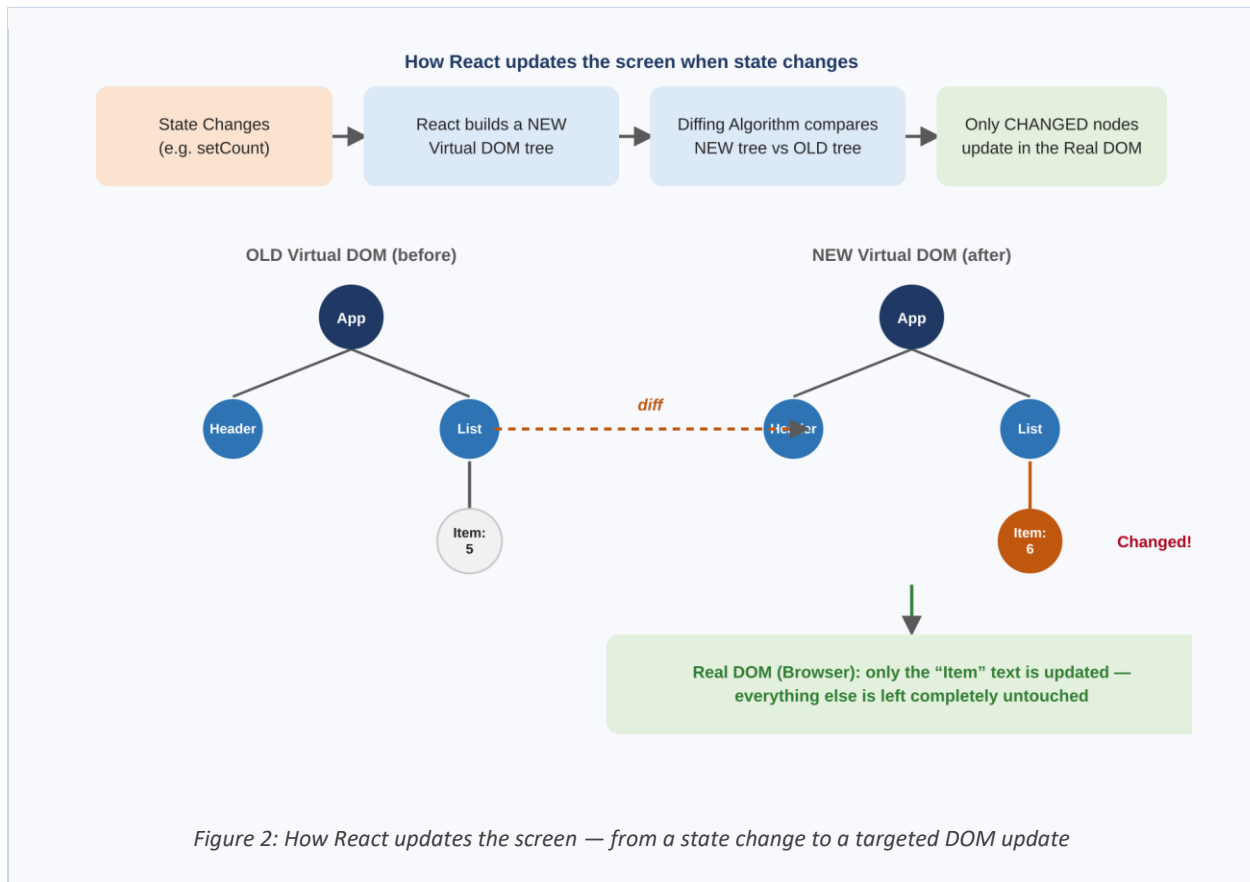


5. Efficient DOM Updates This entire process—working with the Virtual DOM, diffing, and selective updating—makes the UI rendering much faster and more efficient than manipulating the Real DOM directly.

1.2.3 The React Fiber Reconciliation Engine & The Diffing Algorithm

Fiber is React's internal engine that manages rendering work. Whenever state changes, React uses Fiber to build a new Virtual DOM tree and then runs a process called reconciliation: it compares (diffs) the new tree against the previous one, node by node.

The diffing algorithm figures out the smallest possible set of changes needed, and only THOSE specific changes get applied to the Real DOM. This is exactly why React is fast: it avoids unnecessary, expensive updates to the actual page.



Example Syntax

```

// You never call the diffing algorithm yourself -> it runs automatically
// behind the scenes whenever you call a state updater function, e.g.:
setItemCount(6); // triggers: new VDOM tree -> diff -> targeted DOM update
  
```

*Beginner Tip: You will rarely think about Fiber or diffing directly while coding. What matters as a beginner is the **RESULT**: React only updates the parts of the page that actually changed.*

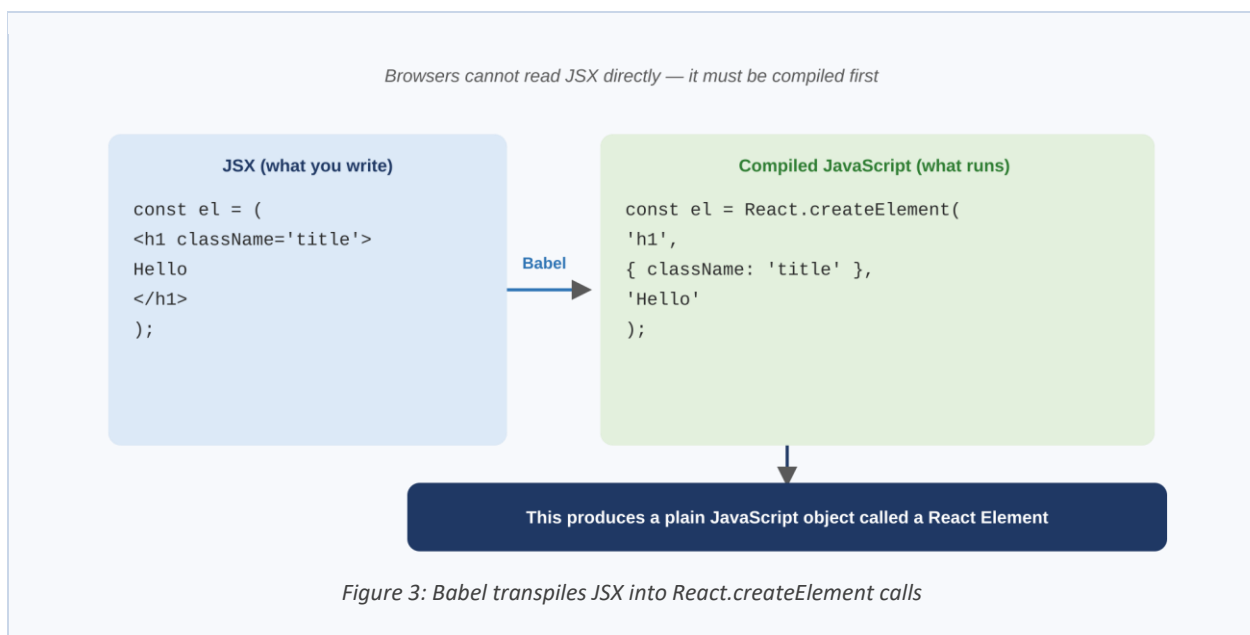
2. JSX (JavaScript XML) Fundamentals

2.1 Compilation & Compilation Tools

2.1.1 The Role of Transpilers (Babel) in Converting JSX to React.createElement

JSX looks like HTML mixed into JavaScript, but browsers cannot actually read JSX directly. A tool called Babel transpiles (translates) your JSX code into plain JavaScript function calls, specifically calls to `React.createElement`, before the code ever reaches the browser.

Understanding this helps explain many JSX rules: since JSX becomes a function call, it must follow JavaScript's rules for function calls and expressions, not HTML's more relaxed rules.



Example Syntax

```
// What you write (JSX)
const el = <h1 className='title'>Hello</h1>;

// What Babel turns it into (plain JavaScript)
const el = React.createElement('h1', { className: 'title' }, 'Hello');
```

2.1.2 The Modern JSX Runtime Transformation

Older versions of React required you to manually import React in every single file that used JSX, because JSX secretly compiled down to `React.createElement` calls. Modern tooling uses a newer JSX runtime that automatically imports the small helper functions it needs behind the scenes, so you usually do not need to write `import React from 'react'` anymore just to use JSX.

Example Syntax

```
// Older JSX runtime (classic) required this in every file using JSX:
import React from 'react';
function App() {
  return <h1>Hello</h1>;
}

// Modern JSX runtime handles this automatically behind the scenes,
// so this import is often no longer required just for JSX to work.
```

2.2 Syntactic Strict Rules

2.2.1 The Single Root Element Constraint & React Fragments (<>...</>)

A component's JSX must return exactly ONE root (outermost) element, because JSX compiles down to a single function call, and a function can only return one value. If you need multiple sibling elements without adding an extra wrapper <div> to the actual page, you can wrap them in a React Fragment instead, written as <>...</>.

Example Syntax

```
// This is INVALID -> two sibling root elements
// function App() {
//   return (
//     <h1>Title</h1>
//     <p>Paragraph</p>
//   );
// }

// This is VALID -> wrapped in a Fragment, no extra DOM element added
function App() {
  return (
    <>
      <h1>Title</h1>
      <p>Paragraph</p>
    </>
  );
}
```

2.2.2 Mapping HTML Attributes to camelCase JavaScript Properties

Because JSX ultimately becomes JavaScript, many familiar HTML attributes are renamed to camelCase, and a few use different names entirely to avoid clashing with reserved JavaScript keywords. The two most common examples are class becoming className, and the for attribute on labels becoming htmlFor.

Example Syntax

// HTML	// JSX
---------	--------

```
// <div class='box'>           <div className='box'>
// <label for='name'>         <label htmlFor='name'>
// <input onclick='...'>      <input onClick={handleClick}>

function Box() {
  return <div className='box'>Content</div>;
}
```

2.2.3 Dynamic Evaluation: Evaluating JavaScript Expressions with Curly Braces {}

Curly braces { } let you drop straight into JavaScript from inside JSX. Anything that is a valid JavaScript EXPRESSION (something that produces a value, like a variable, a function call, or a ternary) can go inside them. Statements like if or for cannot go directly inside curly braces, because they do not produce a value.

Example Syntax

```
function Greeting({ name }) {
  const hour = new Date().getHours();
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>{hour < 12 ? 'Good morning' : 'Good afternoon'}</p>
      <p>Two plus two is {2 + 2}</p>
    </div>
  );
}
```

3. React Functional Components

3.1 Component Architecture

3.1.1 Defining Components as Functional Elements

A functional component is simply a regular JavaScript function that returns JSX describing what should appear on the screen. By convention, component names always start with a capital letter, which is how React tells them apart from regular HTML tags.

Example Syntax

```
function Welcome() {  
  return <h1>Welcome to React!</h1>;  
}  
  
// Using it elsewhere, just like an HTML tag:  
function App() {  
  return <Welcome />;  
}
```

3.1.2 Strict Purity Contracts: Idempotency & Avoiding Side Effects during Rendering

A React component should be 'pure' during rendering: given the same props and state, it should always return the same JSX, a property called idempotency. Components should never change any outside variable, mutate props, or trigger things like network requests directly inside the function body while rendering.

Actions like fetching data, changing a global variable, or starting a timer are called side effects, and they belong inside the `useEffect` hook (covered later), not directly inside the component's main render logic.

Example Syntax

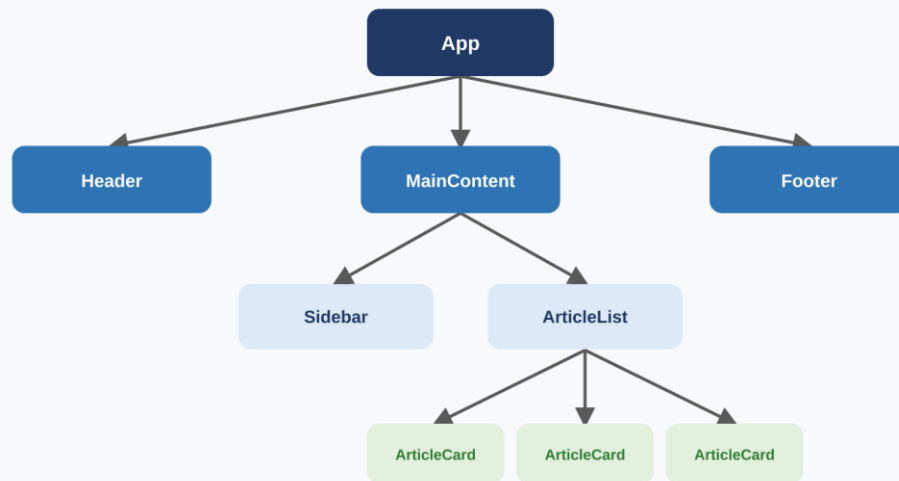
```
// Bad -> a side effect happening directly during render  
let renderCount = 0;  
function BadComponent() {  
  renderCount++; // mutating something outside the component -> avoid this  
  return <p>Rendered {renderCount} times</p>;  
}  
  
// Good -> rendering is pure, based only on props/state passed in  
function GoodComponent({ count }) {  
  return <p>Rendered {count} times</p>;  
}
```

Beginner Tip: If your component's render logic reads or changes something outside of its own props and state, that is usually a sign it needs a `useEffect` instead.

3.2 Modular Composition

3.2.1 Component Nesting, Isolation, and Reusability Models

Real applications are built by nesting components inside other components, forming a tree, much like the HTML tag tree itself. Each component is isolated, meaning it manages its own logic and does not need to know how its parent or siblings are implemented, which makes each piece reusable in different places.



Each box is an independent, reusable component nested inside its parent — this nesting is what “composition” means in React.

Figure 4: A typical component composition tree

Example Syntax

```

function ArticleCard({ title }) {
  return <div className='card'>{title}</div>;
}

function ArticleList({ articles }) {
  return (
    <div>
      {articles.map(a => <ArticleCard key={a.id} title={a.title} />)}
    </div>
  );
}

function MainContent() {
  return (
    <div>
      <Sidebar />
      <ArticleList articles={[{ id: 1, title: 'Post One' }] } />
    </div>
  );
}
  
```

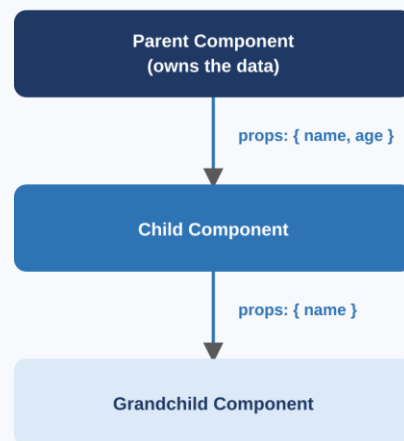
```
);  
}
```

4. Data Mechanics: Props & State

4.1 Props (Properties) Deep Dive

4.1.1 The Unidirectional Data Flow Model (Top-to-Bottom)

Props (short for properties) are how a parent component passes data down to a child component, similar to arguments passed into a function. Data in React always flows in ONE direction: downward, from parent to child. A child can never directly send data back up to its parent through props.



One-Way Street
Props always flow
DOWNWARD only.

A child can never send
props back up to its
parent directly.

Figure 5: Props always flow downward, from parent to child to grandchild

Example Syntax

```
function Parent() {  
  return <Child name='Alice' age={25} />;  
}  
  
function Child({ name, age }) {  
  return <p>{name} is {age} years old</p>;  
}
```

4.1.2 Read-Only Immutability Constraints of Props

Props are read-only inside the component that receives them. A component must never try to change its own props directly; if the data needs to change, that change should happen in the parent (usually through state), which then passes new props back down.

Example Syntax

```
function Child({ age }) {  
  // age = age + 1; // Wrong! Never reassign a prop directly  
  return <p>Age: {age}</p>;  
}  
// If age needs to change, the PARENT should update its own state,  
// which will automatically pass a new 'age' prop down to Child.
```

4.1.3 Destructuring and Setting Default Props

Instead of writing props.name every time, most React code deconstructs props directly in the function's parameter list. You can also assign a default value right there, which is used automatically whenever that prop is not supplied by the parent.

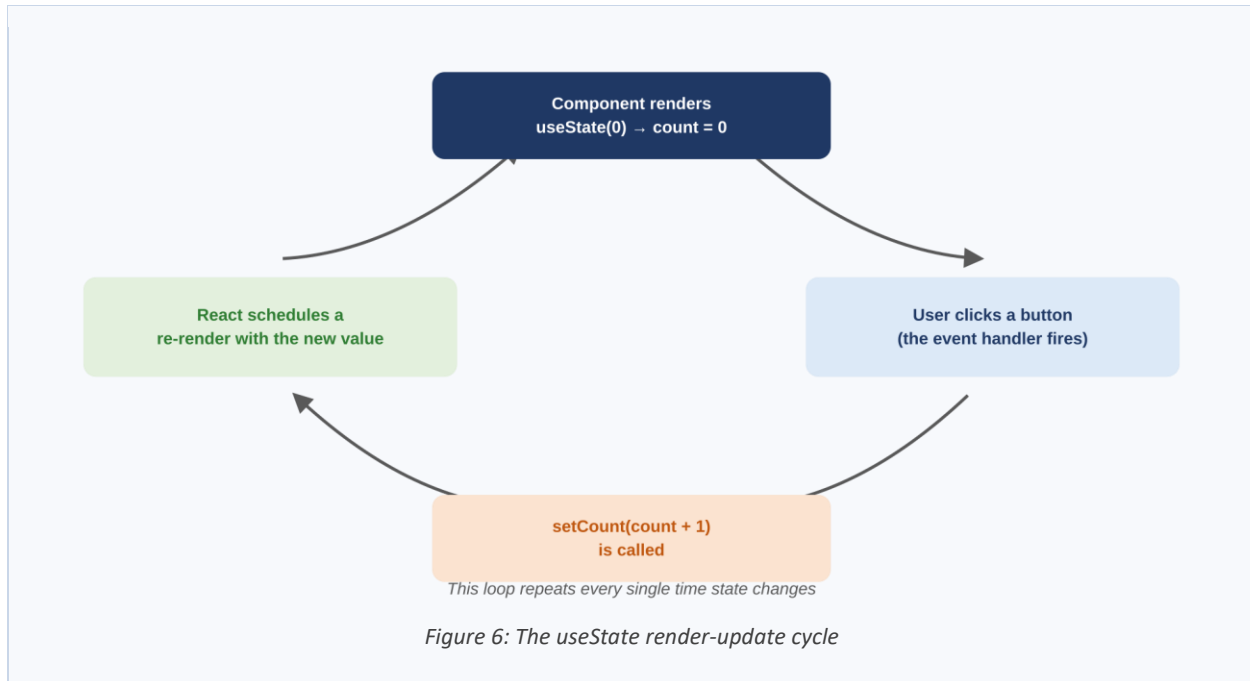
Example Syntax

```
// Without destructuring  
function Greeting(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
// With destructuring and a default value  
function Greeting({ name = 'Guest' }) {  
  return <h1>Hello, {name}</h1>;  
}  
  
<Greeting />;           {/* renders: Hello, Guest */}  
<Greeting name='Alice' />; {/* renders: Hello, Alice */}
```

4.2 State Management Foundations

4.2.1 Local Component Memory & The useState Hook Engine

State is data that a component keeps track of internally, and that can change over time, usually as a result of user interaction. The useState hook gives a component its own private 'memory': it returns the current value, plus a function you call to update that value and trigger a re-render.



Example Syntax

```

import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
}
  
```

4.2.2 Asynchronous State Updates & Batching Behaviors

Calling a state setter function does not update the variable instantly; the change is scheduled, and React batches multiple state updates together for better performance, applying them all at once before the next re-render. This is why logging a state variable immediately after calling its setter often still shows the OLD value.

Example Syntax

```

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }
}
  
```

```
    console.log(count); // still logs the OLD value -> update is not instant
  }

  // If you need to update based on the PREVIOUS state reliably,
  // pass a function to the setter instead:
  function handleClickSafely() {
    setCount(prevCount => prevCount + 1);
  }

  return <button onClick={handleClickSafely}>Count: {count}</button>;
}
```

Beginner Tip: When a new state value depends on the previous one, always use the function form, like `setCount(prev => prev + 1)`, instead of `setCount(count + 1)`.

4.2.3 Updating Objects and Arrays State Immutably

When state holds an object or an array, you must never mutate it directly (for example, by pushing into an array or changing a property in place). React compares old and new state by reference, so it will not notice an in-place change. Instead, always create a brand new object or array, usually with the spread operator, and pass that new copy to the state setter.

Example Syntax

```
const [user, setUser] = useState({ name: 'Alice', age: 25 });

// Wrong -> mutates the existing object directly
// user.age = 26;
// setUser(user); // React may not detect this as a real change

// Correct -> create a new object with the spread operator
setUser({ ...user, age: 26 });

const [items, setItems] = useState([1, 2, 3]);

// Wrong -> mutates the existing array directly
// items.push(4);

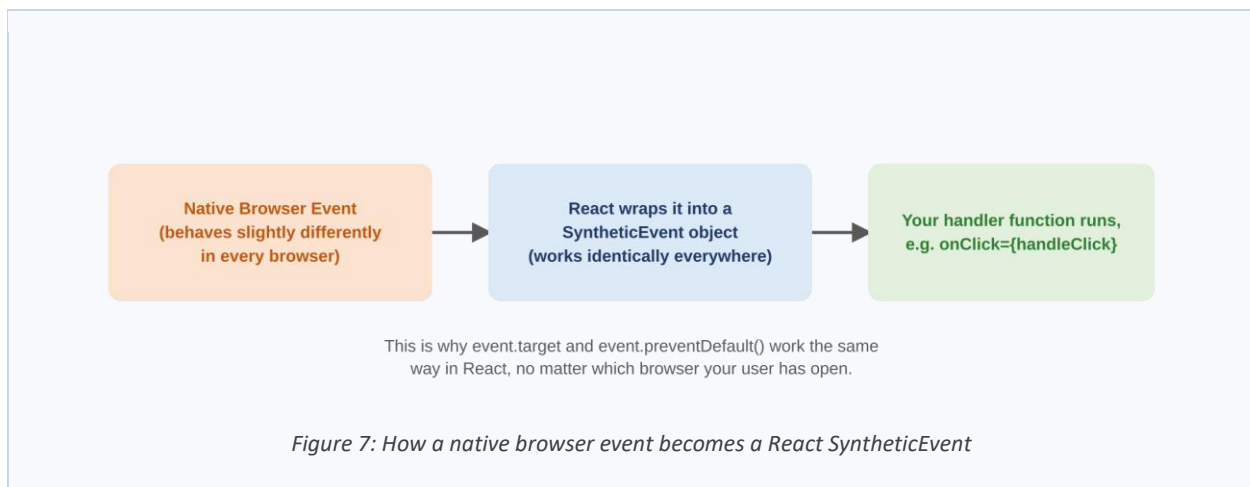
// Correct -> create a new array
setItems([...items, 4]);
```

5. Interactive Event Handling & Conditional Layouts

5.1 Event System Architecture

5.1.1 Synthetic Events: Normalizing Browser Compatibility Cross-Platform

Different browsers can implement native events slightly differently. React solves this by wrapping every native browser event inside its own `SyntheticEvent` object, which behaves consistently no matter which browser your user has open. You use it exactly like a normal DOM event, for example reading `event.target.value`.



Example Syntax

```
function SearchBox() {
  function handleChange(event) {
    console.log(event.target.value); // works the same in every browser
  }
  return <input onChange={handleChange} />;
}
```

5.1.2 Passing Reference Handlers vs. Immediate Function Invocation Footguns

When attaching an event handler, you must pass a REFERENCE to the function, not call it immediately. Writing `onClick={handleClick}` passes the function itself, to be called later on click. Writing `onClick={handleClick()}` calls the function immediately during render, which is almost never what you want, and can even cause an infinite render loop if that function updates state.

Example Syntax

```
function handleClick() {
  console.log('Button clicked');
}
```

```
// Correct -> passes a reference, runs only when clicked
<button onClick={handleClick}>Click me</button>;

// Wrong -> calls handleClick() immediately during render, not on click!
<button onClick={handleClick()}>Click me</button>;

// If you need to pass an argument, wrap it in an inline arrow function:
<button onClick={() => handleClick('extra info')}>Click me</button>;
```

Beginner Tip: A quick mental check: if there is a () after the function name inside onClick={...}, ask yourself whether you actually want it to run immediately during render. Usually, you do not.

5.2 Dynamic Conditional Rendering Patterns

5.2.1 Inline Expressions: Ternary Operators (? :) for Binary Branches

When you need to choose between exactly two possible outputs, a ternary operator inside curly braces is the standard way to do it directly inside JSX, since JSX can only hold expressions, not full if statements.

Example Syntax

```
function StatusMessage({ isOnline }) {
  return (
    <p>
      {isOnline ? 'User is online' : 'User is offline'}
    </p>
  );
}
```

5.2.2 Short-Circuit Operators (&&) for Short-Form Evaluation Guard Rails

When you only need to show something in ONE case, and show nothing at all otherwise, the && operator is a shorter alternative to a ternary. Remember the earlier quirk: if the left-hand value could be the number 0, convert it to a real boolean first (for example with > 0), otherwise React will render a stray 0 on the screen.

Example Syntax

```
function Notifications({ count }) {
  return (
    <div>
      {count > 0 && <span>You have {count} new messages</span>}
    </div>
  );
}
// Using {count && ...} instead could render a 0 on screen when count is 0
```

6. Iterative Lists & The Key Constraint

6.1 Rendering Data Collections

6.1.1 Projecting Arrays to JSX Elements via .map()

To display a list of data, you use the array's `.map()` method to transform each piece of data into a JSX element, producing a new array of elements that React can render directly.

Example Syntax

```
function FruitList({ fruits }) {  
  return (  
    <ul>  
      {fruits.map(fruit => (  
        <li key={fruit.id}>{fruit.name}</li>  
      ))}  
    </ul>  
  );  
}
```

6.2 The Key Attribute Contract

6.2.1 Why Keys Matter: Optimizing the Virtual DOM Diffing Engine

The key prop gives React a stable identity for each item in a list, so its diffing algorithm can tell exactly which items were added, removed, or reordered, instead of having to guess. This makes updates faster and more accurate, and it also protects each list item's internal state from getting mixed up with another item's.

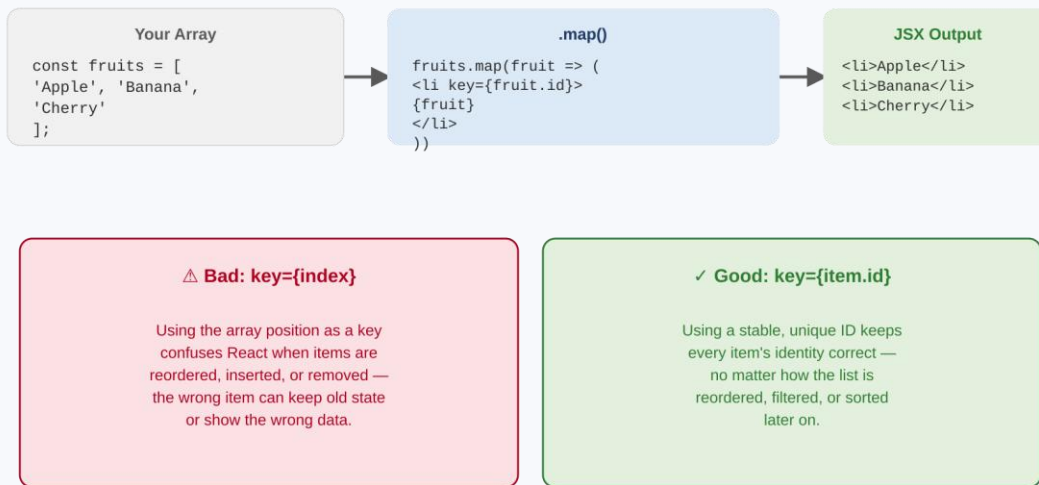


Figure 8: Turning an array into JSX with `.map()`, and why the key you choose matters

Example Syntax

```
const fruits = [
  { id: 'f1', name: 'Apple' },
  { id: 'f2', name: 'Banana' },
];

fruits.map(fruit => <li key={fruit.id}>{fruit.name}</li>);
```

6.2.2 The Array Index Anti-Pattern and Stable Identity Generation

It is tempting to use the array's index as the key, and React will not stop you, but this is risky whenever the list can be reordered, filtered, sorted, or have items inserted or removed. Because the index shifts around whenever the list changes shape, React can end up matching the wrong old item to the wrong new position, which can show incorrect data or lose input values and other component state.

The fix is to always use a stable, unique identifier that belongs to the data itself, such as a database ID, and never regenerate it on every render.

Example Syntax

```
// Risky -> using the index as a key
fruits.map((fruit, index) => <li key={index}>{fruit.name}</li>);

// Safe -> using a stable, unique ID from the data itself
fruits.map(fruit => <li key={fruit.id}>{fruit.name}</li>);

// If your data truly has no ID yet, generate one ONCE when the
// item is created, for example: { id: crypto.randomUUID(), ... }
// Never generate a new ID inside the render / map function itself.
```

7. Component Lifecycle Concepts & Coordination

7.1 The Core Lifecycle Phases

7.1.1 Mount (Initialization & DOM Insertion)

Mounting is the very first phase of a component's life: React creates the component, runs its function body for the first time, and inserts the resulting elements into the Real DOM for the very first time.

Example Syntax

```
function Welcome() {  
  console.log('Rendering...'); // runs once, the first time it mounts  
  return <h1>Welcome!</h1>;  
}
```

7.1.2 Update (Prop/State Change Re-rendering)

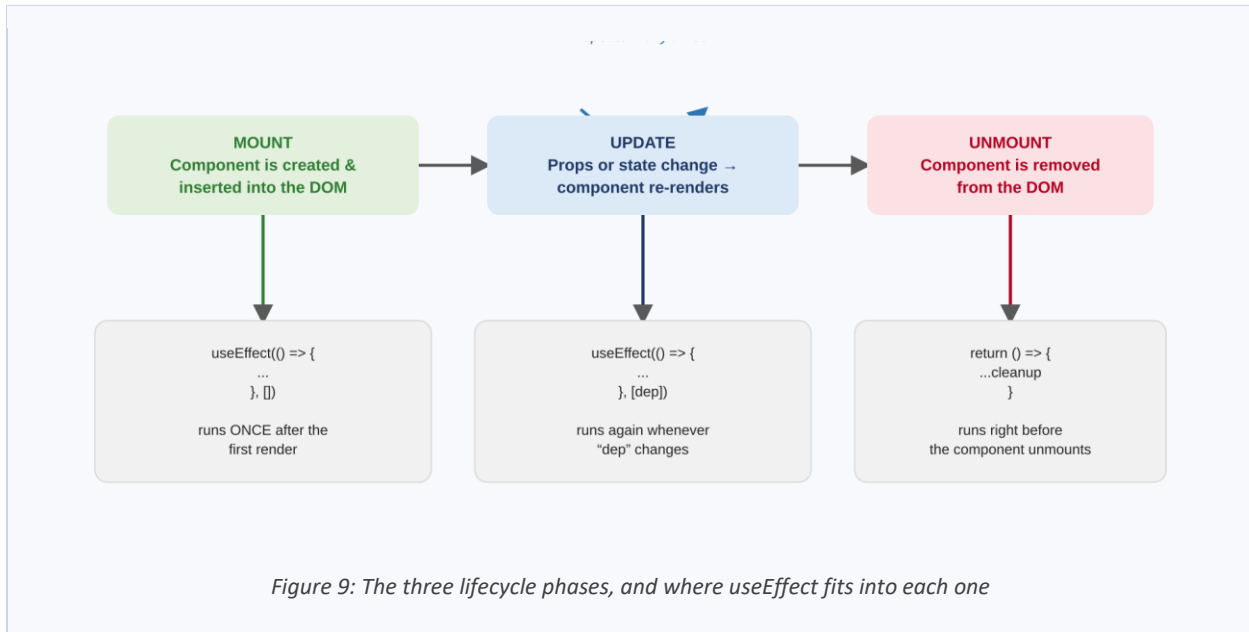
After mounting, a component can re-render many times throughout its life, any time its props change, its own state changes, or its parent re-renders. Each of these re-renders is called an update.

Example Syntax

```
function Counter() {  
  const [count, setCount] = useState(0);  
  console.log('Rendering with count =', count); // logs on every update too  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

7.1.3 Unmount (DOM Clean-up & Destruction)

Unmounting is the final phase: the component is removed from the screen entirely, for example when a user navigates away or a parent stops rendering it. This is the moment to clean up anything the component started, like timers, subscriptions, or event listeners, so they do not keep running after the component is gone.



Example Syntax

```
// Cleanup is registered inside useEffect (covered next), by
// returning a function from it -> that returned function runs
// automatically right before the component unmounts.
```

7.2 Lifecycle Synchronization

7.2.1 Managing Lifecycles using the `useEffect` Hook

`useEffect` lets a function component run side effects, meaning code that reaches outside of pure rendering, such as fetching data, subscribing to something, or manually changing the document title. It runs automatically after the component has rendered and the DOM has been updated.

Example Syntax

```
import { useEffect, useState } from 'react';

function PageTitleUpdater({ title }) {
  useEffect(() => {
    document.title = title; // a side effect -> changing something outside React
  });

  return <p>Check the browser tab title!</p>;
}
```

7.2.2 The Dependency Array (`[]`): Control-Flow Triggers

The second argument to `useEffect` is the dependency array, and it controls WHEN the effect re-runs. An empty array `[]` means the effect runs only once, right after the first mount. Listing variables inside the

array, like [count], means the effect re-runs whenever any of those specific values change. Leaving the array out entirely means the effect runs after every single render.

Example Syntax

```
useEffect(() => {
  console.log('Runs only once, after the first render');
}, []);

useEffect(() => {
  console.log('Runs whenever count changes');
}, [count]);

useEffect(() => {
  console.log('Runs after every single render, no matter what');
});
```

7.2.3 Clean-up Callbacks: Preventing Memory Leaks and Subscriptions Drops

If you return a function from inside `useEffect`, React treats that returned function as a cleanup callback. It runs automatically right before the component unmounts, and also right before the effect runs again on a later update. This is the correct place to cancel timers, remove event listeners, or close subscriptions, preventing memory leaks.

Example Syntax

```
useEffect(() => {
  const timerId = setInterval(() => {
    console.log('Tick');
  }, 1000);

  // Cleanup function -> runs before unmount (or before the effect re-runs)
  return () => {
    clearInterval(timerId);
  };
}, []);
```

Beginner Tip: A good rule of thumb: if your effect starts something (a timer, a subscription, an event listener), it almost always needs a matching cleanup function to stop it.